TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

Marks: 4

11.1 Recommended Final Design

For the metro smart-card system, I recommend using object-oriented programming with suitable data types, constants and classes.

Component	Recommended Design	Reason
Fare	long long	Stores money accurately in paise
Balance	long long	Supports large balances and totals
Zone	enum class	Represents different metro zones
Peak hour	bool	Stores true or false
Journey count	long long	Supports millions of journeys
Fare values	constexpr	Keeps fixed values in one place
SmartCard	Class	Protects the card balance
Journey	Class	Stores journey details
FareTable	Class	Maintains fare information

11.2 How Fare Revisions Will Be Made

Fare values should be stored in one common location using constants. For example:

```
constexpr int INNER_FARE = 1000;\nconstexpr int MIDDLE_FARE = 2000;\nconstexpr int OUTER_FARE = 30
```

The fare values are stored in paise. If the metro authority changes a fare, the corresponding constant can be updated in one place. This avoids changing the same fare value in multiple parts of the program.

11.3 Limitations of the Design

• The initial design supports only three zones.

• The student and senior citizen concession percentages are not specified in the assignment.

• When constants are used, a fare revision requires updating and recompiling the program.

• The basic program does not include database storage or transaction history.

• It does not handle multiple metro gates accessing the same card simultaneously.

These limitations can be addressed by adding new classes and improving the system when requirements change.

11.4 Whether the Design Supports the Whole Network

The proposed design can support the entire metro network because it uses suitable data types and simple fare calculation methods. The assignment mentions 20,000 journeys per day at one station, 500,000 per day on one line and 3,000,000 per day across the whole network.

Assuming every journey costs Rs. 30, the total for 3,000,000 journeys is 3,000,000 × 3000 paise = 9,000,000,000 paise. This exceeds the maximum value of a typical 32-bit int, so long long should be used.

Each journey requires a fixed number of fare calculations and balance checks. Time complexity per journey is O(1). For n journeys, total processing time is O(n).

Therefore, the design is suitable for fare calculations across the whole network. A real metro system would also require secure storage and transaction management.

11.5 Alternative Concept – Configuration File

Instead of storing fare values directly in the program, they can be stored in an external configuration file. For example, a file named fare.txt can contain:

```
1000\n2000\n3000
```

When the program starts, it reads the fare values from this file.

Advantages: Fare values can be changed without modifying the source code, recompilation is not required for every fare revision, and updated values can be loaded at start-up.

Disadvantages: The file must be available, invalid values may cause incorrect calculations, and extra code is needed to read and validate the file.

Complexity analysis: If there are z zones, reading all fare values takes O(z) time. The memory required to store the fares is O(z). Constants are simpler for a small system, whereas a configuration file is more flexible when fares change frequently.

11.6 Final Recommendation

I recommend retaining the object-oriented design with some improvements. The SmartCard class should protect the balance, while the FareTable class should maintain the fare values. Integer paise avoids rounding errors, and long long supports large network collections.

The design can be extended when new zones and fare rules are introduced.

Conclusion: The object-oriented approach with suitable data types, constants and classes provides an accurate, simple and maintainable solution for the metro smart-card fare system. It is suitable for the current requirements and can be modified for future improvements.